

QUBIT Wind Farm Formulation

Dhiaeddine Zarouan, Aymen Gouja

April-Mai 2026

1 Problem Statement

We have an offshore Wind farm, containing 46 Wind turbines and 2 collectors. Each collector can handle up to 23 turbines. Connecting this network with cables having different diameters.

- D_1 - 500 euro/meter can handle the power generated by 1 Turbine
- D_2 - 700 euro/meter can handle the power generated by 2 Turbines
- D_3 - 900 euro/meter can handle the power generated by 3 Turbines
- D_4 - 1100 euro/meter can handle the power generated by 4 Turbines

Each turbine can at most have 2 connections, either Turbine-Turbine + Turbine-Collector or Turbine-Turbine + Turbine-Turbine. The cables shouldn't create loops.

Objective: Minimize the cost of installing and connecting all cables.

2 Chosen Formulation

2.1 Variable

we decided to consider x_{ij}^k as our variable with:

$$x_{ij}^k = \begin{cases} 1 & : \text{Cable from } E_i \text{ to } E_j \text{ with diameter } k, E \text{ being element} \\ 0 & : \text{No cable} \end{cases}$$

The variables i , j , and k take the following values:

- $i \in [1..48]$
- $j \in [1..48]$

- $k \in [1..4]$

we also consider C_i or C_j to be a collector if $i = 47$ or $i = 48$ and j playing a symmetric role.

2.2 Cost function

As we're minimizing the cost of installing all cables in the network, it only makes sense to minimize this function with the given constraints.

$$J = \sum_k \sum_i \sum_j w_k * d_{ij} * x_{ij}^k + (\sum \lambda_i P_i(X)) \quad (1)$$

with w_k being the cost of cable with diameter k (per meter) and d_{ij} the length of said cable between Element i and j .

$P(X)$ being the penalty functions and λ the penalty coefficient.

2.3 Constraints

First constraints are the 23-turbine limit to each collector.

$$\sum_k \sum_j k * x_{c_1 t_j}^k = 23 \quad (2)$$

$$\sum_k \sum_j k * x_{c_2 t_j}^k = 23 \quad (3)$$

These two equations give the requirement to connect 23 ($N/2$) turbines to each collector. with c_1 and c_2 being the indices of the collectors and t_j the indices of turbines; $t_j \in [1..N-2]$ Then we have the turbine connections limit.

$$\forall j, \sum_k \sum_i k * x_{ij}^k - \sum_k \sum_i k * x_{ji}^k = 1 \quad (4)$$

Generally, this constraint represents the power flow in each turbine, which helps with the cable width.

Example: For a turbine i , we check all cables flowing in with their respective widths k and the cable flowing out with their respective widths k and the result of their difference should be 1, as in $k * x_{ij}^k - (k-1) * x_{i,j+1}^{k-1} = 1$ After much trial and error, we opted to add a couple of constraints.

$$\sum_i \sum_j \sum_k x_{ij}^k = 46 \quad (5)$$

The equation above translates to the need for only 46 connections (the number of turbines needed to connect)

$$\sum_k \sum_i x_{ii}^k = 0 \quad (6)$$

This constraint restricts each turbine from connecting to itself since the solver finds self-connecting to be much cheaper in cost compared to any other.¹

$$\forall i \forall k, \sum_k x_{ij}^k = 1 \quad (7)$$

The constraint above forces a connection from turbine i to turbine j to have only one cable type, since satisfying the fourth constraint allows two close turbines to connect many times.

2.4 QUBO Vector

Since we are required to work in the QUBO formulation, which minimizes any $y^T Q y$, our variable should be transformed into a vector y .

And for each y_t we associated one x_{ij}^k such as:

$$i = \left\lfloor \frac{t}{n} \right\rfloor \bmod n + 1 \quad (8)$$

$$j = t \bmod n + 1 \quad (9)$$

$$k = \left\lfloor \frac{t}{n^2} \right\rfloor + 1 \quad (10)$$

2.5 QUBO Constraints

2.5.1 Quadratic Form

Each constraint will transform into a penalty function. Equation 2 and 3 will be in the form:

$$P_1(X) = \left(\sum_k \sum_j k * x_{c_1 t_j}^k - 23 \right)^2 \quad (11)$$

$$P_2(X) = \left(\sum_k \sum_j k * x_{c_2 t_j}^k - 23 \right)^2 \quad (12)$$

¹This will be later proved ineffective

and Equation 4 will be:

$$P_3(X) = \sum_i (\sum_k \sum_j k * (x_{ij}^k - x_{ji}^k) - 1)^2 \quad (13)$$

Equation 5:

$$P_4(X) = (\sum_i \sum_j \sum_k x_{ij}^k - 46)^2 \quad (14)$$

Equation 6:

$$P_5(X) = (\sum_i \sum_k x_{ii}^k)^2 \quad (15)$$

Equation 7:

$$P_6(X) = \sum_i \sum_j (\sum_k x_{ij}^k - 1)^2 \quad (16)$$

Going back to the fifth penalty $P_5(X)$, it is proven difficult to satisfy it, however high the coefficient λ_5 so we decided to remove it in favor of changing the distance matrix.

Distance Matrix In our implementation, we found that creating a distance matrix at startup reduces the number of calculations.

The distance matrix is defined as such :

$D_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$ with x_i and y_i the coordinates of turbine i, not to be confused with x and y the components of vectors X and Y, respectively, our chosen formulation variable.

Removal of Penalty P_5 Since $D_{ii} = 0$ always, the cost of connecting a turbine to itself is non-existent. So when removing the penalty, we set $D_{ii} = 2 * \max(D)$ allowing each turbine to favor others instead of itself.

2.5.2 Developping Quadratic Form

Now to get the Matrix form for this problem ($y^T Q y$) we need to develop each penalty function and get relations between components of X and then relate them to Y.

Starting with Equation 4:

$$(\sum_k \sum_j k * (x_{ij}^k - x_{ji}^k) - 1)^2 = (\sum_k \sum_j k (x_{ij}^k - x_{ji}^k))^2 + 1 - 2(\sum_k \sum_j k (x_{ij}^k - x_{ji}^k))$$

Taking the first term on its own:

$$\begin{aligned}
(\sum_k \sum_j k x_{ij}^k - x_{ji}^k)^2 &= \sum_k k^2 \left[\sum_j (x_{ij}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ij}^k * x_{ij'}^k \right] \\
+ 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'} &+ \sum_k k^2 \left[\sum_j (x_{ji}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ji}^k * x_{ji'}^k \right] \\
+ 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ji}^k * x_{ji'}^{k'} &- 2 \sum_k \sum_{k'} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'}
\end{aligned} \tag{17}$$

Now the entire Penalty function corresponding to this constraint will be:

$$\begin{aligned}
P_3(X) = & \sum_i \left[\sum_k k^2 \left[\sum_j (x_{ij}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ij}^k * x_{ij'}^k \right] \right. \\
& + 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'} \\
& + \sum_k k^2 \left[\sum_j (x_{ji}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ji}^k * x_{ji'}^k \right] \\
& + 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ji}^k * x_{ji'}^{k'} \\
& - 2 \sum_k \sum_{k'} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'} \\
& \left. + 1 - 2(\sum_k \sum_j k(x_{ij}^k - x_{ji}^k)) \right]
\end{aligned} \tag{18}$$

And then $P_1(X)$ and $P_2(X)$ will have the same form with $\mathbf{a}$ being either equal to 1 or 2:

$$\begin{aligned}
P_a(X) = & \sum_k k^2 \left[\sum_j (x_{cat_j}^k)^2 + 2 \sum_j \sum_{j' < j} x_{cat_j}^k * x_{cat_{j'}}^k \right] \\
& + 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{cat_j}^k * x_{cat_{j'}}^{k'} \\
& + 23^2 - 46 \sum_k \sum_j k * x_{cat_j}^k
\end{aligned} \tag{19}$$

Finally, the cost function will be:

$$\begin{aligned}
J(X) = & \sum_k \sum_i \sum_j w_k * d_{ij} * x_{ij}^k \\
+ & \lambda_1 \sum_i [(\sum_k k^2 [\sum_j (x_{ij}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ij}^k * x_{ij'}^k]) \\
+ & 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'} \\
+ & \sum_k k^2 [\sum_j (x_{ji}^k)^2 + 2 \sum_j \sum_{j' < j} x_{ji}^k * x_{j'i}^k] \\
+ & 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{ji}^k * x_{j'i}^{k'} \\
- & 2 \sum_k \sum_{k'} \sum_j \sum_{j'} k * k' * x_{ij}^k * x_{ij'}^{k'} \\
+ & 1 - 2(\sum_k \sum_j k(x_{ij}^k - x_{ji}^k))] \\
+ & \lambda_2 \sum_k k^2 [\sum_j (x_{c_1 t_j}^k)^2 + 2 \sum_j \sum_{j' < j} x_{c_1 t_j}^k * x_{c_1 t_{j'}}^k] \\
+ & 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{c_1 t_j}^k * x_{c_1 t_{j'}}^{k'} \\
+ & 23^2 - 46 \sum_k \sum_j k * x_{c_1 t_j}^k \\
+ & \lambda_3 \sum_k k^2 [\sum_j (x_{c_2 t_j}^k)^2 + 2 \sum_j \sum_{j' < j} x_{c_2 t_j}^k * x_{c_2 t_{j'}}^k] \\
+ & 2 \sum_k \sum_{k' < k} \sum_j \sum_{j'} k * k' * x_{c_2 t_j}^k * x_{c_2 t_{j'}}^{k'} \\
+ & 23^2 - 46 \sum_k \sum_j k * x_{c_2 t_j}^k \\
+ & \lambda_4 [\sum_i \sum_j \sum_k k^2 * x_{ij}^k \\
+ & \sum_i \sum_j \sum_{k < k'} 2 * k * k' * x_{ij}^k * x_{ij'}^{k'} \\
+ & \sum_i \sum_{j < j'} \sum_k \sum_{k'} k * k' * x_{ij}^k * x_{ij'}^{k'} \\
+ & \sum_{i < i'} \sum_j \sum_{j'} \sum_k \sum_{k'} 2 * k * k' * x_{ij}^k * x_{i'j'}^{k'} \\
- & \sum_i \sum_j \sum_k 92 * k * x_{ij}^k] \\
+ & \lambda_5 [\sum_i \sum_k x_{ii}^k + \sum_{i < i'} \sum_k 2 * x_{ii}^k \\
+ & \sum_i \sum_{k < k'} 2 * x_{ii}^k] \\
+ & \lambda_6 [\sum_i \sum_j [\sum_k x_{ij}^k + \sum_{k < k'} 2 * x_{ij}^k]]
\end{aligned} \tag{20}$$

3 Roadmap & Future Improvements

Going forward, we aim to:

- Calculate the adequate range for each of the λ_i .
- Research *Singular Value Decomposition (SVD)* and study its compatibility and feasibility in our problem statement.